

CSE 341

Programming Languages

Chapter 6 – Data Types

Kapsamlı Çalışma Rehberi

Complete Study Guide with Quiz Solutions & Practice Problems

Gebze Technical University · Department of Computer Engineering

Spring 2026 · Prepared for Exam Study

Chapter : İçindekiler / Table of Contents

1 .	Giriş – Veri Tipleri Nedir? (Introduction to Data Types)	3
2 .	İlkel Veri Tipleri (Primitive Data Types)	3
3 .	Karakter Dizisi Tipleri (Character String Types)	5
4 .	Numaralandırma Tipleri (Enumeration Types)	6
5 .	Dizi Tipleri (Array Types)	7
6 .	İlişkilendirici Diziler (Associative Arrays)	9
7 .	Kayıt Tipleri (Record Types)	10
8 .	Tuple, Liste ve Union Tipleri	10
9 .	İşaretçi ve Referans Tipleri (Pointer & Reference Types)	11
10 .	Tip Denetimi ve Güçlü Tipleme (Type Checking & Strong Typing)	13
11 .	Tip Denkliği (Type Equivalence)	14
12 .	Review Questions – Tüm Sorular ve Çözümleri	15
13 .	Problem Set – Tüm Sorular ve Çözümleri	21
14 .	Quizlere Benzer Pratik Sorular (Practice Quiz Questions)	25

Chapter 1: Giriş - Veri Tipleri

Introduction to Data Types

Temel Kavramlar

Bir **veri tipi** (data type), bir dizi veri değeri ve bu değerler üzerinde tanımlı işlemler kümesini belirler. Programlar, veri manipüle ederek sonuç üretir; bu nedenle kullanılan veri tiplerinin gerçek dünya nesneleriyle örtüşmesi kritik önem taşır.

TARİ HSEL

Veri tiplerinin tarihsel gelişimi: Fortran (sadece sayısal tipler) → COBOL (ondalık, kayıt tipleri) → PL/I (hassasiyet tanımları) → ALGOL 68 (temel tipler + esnek yapı operatörleri) → Modern diller (soyut veri tipleri / abstract data types).

Tip Sisteminin Kullanım Amaçları

- **Hata Tespiti:** Tip denetimi (type checking) sayesinde hatalar erken yakalanır.
- **Modülerizasyon:** Modüller arası arayüz tutarlılığı sağlanır.
- **Dokümantasyon:** Tip bildirimleri, programın verisi hakkında bilgi verir.

TANIM

Descriptor (Tanımlayıcı): Bir değişkenin tüm özelliklerinin toplamıdır. Statik özellikler için derleme zamanında, dinamik özellikler için çalışma zamanında bellekte tutulur. Symbol table'ın parçasıdır.

Chapter 2: İlkel Veri Tipleri

Primitive Data Types

2.1 Sayısal Tipler (Numeric Types)

Integer (Tam Sayı):

Birçok bilgisayarda doğrudan donanım desteği vardır. Java 4 işaretli tam sayı tipi destekler: byte, short, int, long. C++ ve C# işaretli (signed) tam sayıları da destekler.

Temsil	Açıklama	Not
İşaret-Büyüklik (Sign-Magnitude)	İşaret biti + mutlak değer	Aritmetik için uygun değil
İkiye Tümleme (Two's Complement)	Pozitifin mantıksal tersi + 1	Modern bilgisayarlar kullanır
Bire Tümleme (One's Complement)	Mutlak değer mantıksal tersi	Çift sıfır problemi var

Floating-Point (Kayan Noktalı):

Gerçek sayıları modellemek için kullanılır ancak birçok değer için yalnızca yaklaşım sunar. Modern makineler IEEE 754 standardını kullanır.

Format	İşaret	Üs (Exponent)	Kesir (Fraction)	Toplam
Single (float)	1 bit	8 bit	23 bit	32 bit
Double	1 bit	11 bit	52 bit	64 bit

⚠ DİK KAT 0.1 (ondalık) ikili sistemde sonlu bit sayısı ile tam olarak temsil edilemez: 0.0001100110011... (sonsuz tekrar). Bu, kayan noktalı hesaplamada hata kaynağıdır.

Decimal (Ondalık), Boolean ve Character:

Decimal: İş uygulamaları için BCD (Binary Coded Decimal) formatında saklanır. Ondalık değerleri tam olarak temsil edebilir (float aksine). COBOL, C#, F# destekler. Dezavantajı: üs kullanılamaz, bellek israfı (6 rakam için 24 bit vs 20 bit binary).

Boolean: Sadece true/false değerleri. ALGOL 60'ta tanıtıldı. C/C++'ta sayısal ifadeler koşul olarak kullanılabilir (0=false, sıfır dışı=true). Genellikle 1 byte'ta saklanır.

Character: ASCII (7-bit, 128 karakter), ISO 8859-1 (8-bit, 256 karakter), Unicode/UCS-2 (16-bit), UTF-32 (32-bit). Java Unicode kullanan ilk yaygın dildir.

Chapter 3: Karakter Dizisi Tipleri

Character String Types

Temel Tasarım Soruları

- Stringler özel bir karakter dizisi mi yoksa ilkel tip mi olmalı?
- Stringlerin uzunluğu statik mi yoksa dinamik mi olmalı?

Uzunluk Seçenekleri

Tür	Açıklama	Diller
Static Length (Statik)	String oluşturulduğunda uzunluk sabit	Python, Java String, C++ std::string, Ruby
Limited Dynamic (Sınırlı Dinamik)	Max uzunluk tanımda belirlenir	C, C-style C++
Dynamic Length (Dinamik)	Max yok, heap'te büyür/küçülür	JavaScript, Perl, Python list

C'de String: Önemli Notlar

```
char str[] = "apples";  
  
// Bellekte: a p p l e s \0 (7 eleman – null karakter ile)  
  
strcpy(dest, src); // GÜVENSİZ! dest sınırını aşabilir  
  
// C++ programcıları bunun yerine std::string kullanmalı
```

⚠ GÜVENLİK

C'nin string kütüphanesi (strcpy, strcat, strcmp, strlen) doğası gereği GÜVENSİZDİR. strcpy, hedef dizinin sınırını kontrol etmez ve bu birçok programlama hatasına yol açar. C++ programcıları char dizileri yerine std::string sınıfını kullanmalıdır.

Chapter 4: Numaralandırma Tipleri

Enumeration Types

Numaralandırma tipi, olası tüm değerlerin adlandırılmış sabitler olarak sıralandığı bir tiptir. Okunabilirlik ve güvenilirlik açısından büyük avantaj sağlar.

Tasarım Soruları

- Bir numaralandırma sabiti birden fazla tip tanımında kullanılabilir mi?
- Numaralandırma değerleri tam sayıya dönüştürülür mü (coerced)?
- Başka tipler numaralandırma tipine dönüştürülebilir mi?

Dil Karşılaştırması

Dil	Tam Sayıya Dönüşüm?	Güvenlik	Örnek
C	Evet (otomatik)	Düşük	<code>enum color {red=0, blue=1};</code>
C++	Evet (coerced)	Orta	Sayısal atama cast gerektirir
C#	Hayır	Yüksek	Aritmetik işlem yasak
Java 5+	Hayır	Yüksek	Class olarak tanımlanır
Swift	Hayır (iç değer yok)	Yüksek	<code>enum fruit { case apple }</code>

```
// C++ örneği:
enum colors {red, blue, green, yellow, black};
colors myColor = blue, yourColor = red;
myColor++; // green olur (tam sayıya dönüşüm var!)

// C# örneği (güvenli):
enum days {Mon, Tue, Wed, Thu, Fri, Sat, Sun};
// days d = 3; // HATA! Doğrudan atama yasak
```

Chapter 5: Dizi Tipleri

Array Types

Dizi, aynı tipten elemanların homojen koleksiyonudur. Elemanlara pozisyonları ile erişilir. Referanslar alt simge (subscript/index) ifadeleri kullanılarak belirtilir.

5.1 Dizi Kategorileri (Array Categories)

Kategori	Alt Simge Aralığı	Depolama	Nerede?	Örnek Diller
Static Array	Statik (derleme)	Statik	Data segment	C/C++ static
Fixed Stack-Dynamic	Statik	Çalışma zamanı	Stack	C fonksiyonları
Fixed Heap-Dynamic	Çalışma zamanı	Çalışma zamanı	Heap	Java, C# new[]
Heap-Dynamic	Değişebilir	Değişebilir	Heap	Perl, Python, Ruby

5.2 Erişim Fonksiyonu (Access Function)

Tek boyutlu dizi için erişim fonksiyonu:

```
address(list[k]) = address(list[0]) + k * element_size

// Genel (arbitrary lower bound):
address(list[k]) = address(list[lower_bound]) + (k - lower_bound) * element_size

// 2 boyutlu, row major order:
location(a[i,j]) = address(a[0,0]) + ((i*n + j) * element_size)

// n = satır başına eleman sayısı
```

5.3 Row Major vs Column Major Order

Matris **[3 4 7 / 6 2 5 / 1 3 8]** için:

```
Row Major Order (C, C++, Java, Python):
3, 4, 7, 6, 2, 5, 1, 3, 8 ← satır satır saklanır

Column Major Order (Fortran, MATLAB):
3, 6, 1, 4, 2, 3, 7, 5, 8 ← sütun sütun saklanır
```

5.4 Dilimler (Slices)

```
# Python slice örnekleri:

vector = [2, 4, 6, 8, 10, 12, 14, 16]

vector[3:6] # [8, 10, 12] (indeks 3,4,5)

vector[0:7:2] # [2, 6, 10, 14] (her ikinci eleman)

mat = [[1,2,3],[4,5,6],[7,8,9]]

mat[1] # [4, 5, 6] (2. satır)

mat[0][0:2] # [1, 2]
```


Chapter 6-8: İlişkilendirici Diziler, Kayıtlar, Tuple, Liste, Union

6.1 İlişkilendirici Diziler (Associative Arrays / Hash Tables)

Anahtar-değer (key-value) çiftlerinden oluşan sırasız koleksiyonlardır. Perl'de 'hash', Python'da 'dictionary', Ruby ve Swift'te 'dictionary' olarak bilinir.

```
# Perl (Hash):

%salaries = ("Gary" => 75000, "Perry" => 57000);

$salaries{"Perry"} = 58850; # güncelleme

delete $salaries{"Gary"}; # silme

# Python (Dictionary):

salaries = {"Gary": 75000, "Perry": 57000}

salaries["Perry"] = 58850

del salaries["Gary"]
```

6.2 Kayıt Tipleri (Record Types)

Heterojen veri koleksiyonları için kullanılır. Alanlar (fields) isimlerle tanımlanır. C/C++/C#/Swift'te **struct**, COBOL'de **level numbers** ile tanımlanır.

```
// C struct örneği:

struct Employee {

char name[50];

int id;

float salary;

};

struct Employee e1;

e1.salary = 55000.0; // dot notation

// COBOL field referansı:

// FIRST OF EMPLOYEE-NAME OF EMPLOYEE-RECORD

// ya da kısaltılmış (elliptical): FIRST
```

6.3 Tuple, Liste ve Union Tipleri

Tuple: Kayda benzer ancak eleman isimleri yoktur. Python, ML, F# destekler. Python'da immutable'dır. Çoklu dönüş değeri için kullanılır.

```
# Python tuple:

myTuple = (3, 5.8, 'apple') # farklı tipler olabilir

myTuple[1] # 5.8

# ML tuple:

val myTuple = (3, 5.8, 'apple');

#1(myTuple); # ilk eleman: 3
```

Liste: Fonksiyonel programlamanın temelidir (Lisp). Scheme'de CAR (ilk eleman), CDR (geri kalanı), CONS (yeni eleman ekle) operasyonları kullanılır.

Union (Birleşim): Bir değişkenin farklı zamanlarda farklı tiplerde değer tutmasına olanak tanır. **Free union:** tip denetimi yok (C, C++). **Discriminated union:** tag ile tip takip edilir (ML, Haskell, F#). Java ve C# union içermez (güvenlik odaklı tasarım).

```
// C free union (GÜVENSİZ):

union flexType { int intEl; float floatEl; };

union flexType ell;

ell.intEl = 27;

float x = ell.floatEl; // tip hatası – kontrol edilmez!
```

Chapter 9: İşaretçi ve Referans Tipleri

Pointer & Reference Types

İşaretçi (pointer), bellek adresleri ve nil değerinden oluşan değer aralığına sahip bir tiptir. İki temel kullanım amacı: (1) dolaylı adresleme, (2) dinamik bellek yönetimi.

9.1 Pointer Operasyonları

```
// C/C++ pointer örneği:

int *ptr;

int count, init = 5;

ptr = &init; // & : adres alma operatörü

count = *ptr; // * : dereference (dolaylı erişim) → count = 5

// Pointer aritmetiği:

int list[10];

int *ptr = list; // list[0]'ın adresi

*(ptr + 1) ≡ list[1]

ptr[index] ≡ list[index]
```

9.2 Tehlikeler: Dangling Pointer ve Memory Leak

⚠ TEH LİKE **Dangling Pointer (Sarkan İşaretçi):** Belleği serbest bırakılmış bir heap değişkenine işaret eden pointer. Nedeni: explicit deallocation. Sonuç: Tanımsız davranış, güvenlik açıkları, çöküş.

```
// C++ dangling pointer örneği:

int *arrayPtr1;

int *arrayPtr2 = new int[100];

arrayPtr1 = arrayPtr2;

delete [] arrayPtr2;

// Şimdi arrayPtr1 VE arrayPtr2 dangling pointer!

// Çözüm: delete sonrası ptr = nullptr; (ya da 0)
```

⚠ TEH LİKE **Lost Heap-Dynamic Variable (Memory Leak / Bellek Sızıntısı):** Heap'te tahsis edilmiş ama artık erişilemeyen değişken. p1 yeni bir değişkene işaret ettiğinde, eskisine erişim yolu kaybolur. Garbage (çöp) olarak da bilinir.

9.3 Çözümler: Tombstone ve Locks-and-Keys

Yöntem	Açıklama	Maliyet
Tombstone	Her heap değişkeni için özel tombstone hücresi. Deallocate edilince tombstone nil'e set edilir. Pointer direkt değişkene değil tombstone'a işaret eder.	Yüksek (ekstra bellek + ek indirection)
Locks-and-Keys (UW-Pascal)	Pointer = (key, address) çifti. Heap değişkeni lock değeri tutar. Erişimde key==lock kontrolü yapılır.	Orta (her erişimde karşılaştırma)
Implicit Deallocation (En iyi çözüm)	Programcı bellekten serbest bırakma yapmaz. Runtime system garbage collection yapar. Lisp, Java, C# bu yaklaşımı kullanır.	Düşük (GC overhead)

9.4 Garbage Collection Yöntemleri

Reference Counter (Referans Sayacı - Eager): Her hücrede işaret eden pointer sayısı tutulur. Sayaç sıfırlanınca hücre boşaltılır. Sorunlar: (1) Sayaç için ekstra bellek, (2) Her pointer değişiminde overhead, (3) Dairesel yapılar (circular references) çözülmez.

Mark-Sweep (İşaretle-Süpür - Lazy): Heap dolduğunda tetiklenir. 3 aşama: (1) Tüm hücreler çöp olarak işaretlenir, (2) Erişilebilir hücreler 'kullanımda' olarak işaretlenir (marking phase), (3) İşaretlenmemiş hücreler boşaltılır (sweep phase). Sorun: Uzun duraklamalar (stop-the-world).

9.5 Reference Types (Java, C#)

Referans tipi pointer'a benzer ama adresi değil nesneyi/değeri referans eder. Pointer aritmetiği desteklenmez. Java'da tüm sınıf örnekleri referansla erişilir. Implicit deallocation sayesinde dangling reference oluşamaz.

Chapter 10: Tip Denetimi ve Güçlü Tipleme

Type Checking & Strong Typing

10.1 Type Checking (Tip Denetimi)

Tip denetimi, operandların operatörle uyumlu tipte olmasını sağlama işlemidir. **Uyumlu tip (compatible type)**: ya operatör için yasal tip, ya da örtük dönüşüm (coercion) ile yasal hale gelen tiptir.

**i STAT
IC vs D
YNAMI
C**

Statik tip denetimi: Derleme zamanında yapılır. Daha hızlı, hataları erken yakalar. Statik tip bağlama gerektir. | **Dinamik tip denetimi**: Çalışma zamanında yapılır. Daha esnek ama yavaş ve güvensiz. JavaScript, PHP dinamik tip denetimi kullanır.

10.2 Güçlü Tipleme (Strong Typing)

Bir programlama dili, **tüm tip hataları her zaman tespit ediliyorsa** güçlü tiplidir (strongly typed). Bu, operandların tiplerinin derleme ya da çalışma zamanında belirlenebilmesini gerektirir.

Dil	Strongly Typed?	Açıklama
C	HAYIR	Free union tipler tip denetimi yapılmaz
C++	HAYIR	C union'ları ve uyumsuz cast'ler
Java	NEREDEYSE	Explicit cast hata üretebilir; coercion var
C#	NEREDEYSE	Java'ya benzer; unsafe pointer bölümü var
ML, F#	EVET	Coercion yok, en güvenli
Ada	NEREDEYSE	Derived type ve subtype güvenli tasarım

**⚠ ÖNE
MLİ**

Coercion ve Güçlü Tiplemenin Zayıflaması: Java'da `int + float` işleminde `int` otomatik olarak `float`'a dönüştürülür. Bu, tip denetiminin bir faydasını ortadan kaldırır: `a+d` yerine `a+b` yazılması gereken yerde hata yakalanmaz. ML ve F# coercion yapmaz, dolayısıyla en güvenilir dillerdir.

Chapter 11: Tip Denkliđi

Type Equivalence

İki Temel Yaklaşım

Yaklaşım	Tanım	Avantaj	Dezavantaj
Name Type Equivalence (İsim Denkliđi)	Aynı bildirimde ya da aynı tip ismiyle tanımlanan deđişkenler denkdir	Uygulaması kolay, daha kısıtlayıcı	Tüm tiplerin ismi olmalı; subrange problemi
Structure Type Equivalence (Yapı Denkliđi)	Tiplerin yapısı (structure) özdeşse denkdir	Daha esnek	Uygulaması zor; aynı yapı farklı semantik

Ada'nın Yaklaşımı: Derived Types ve Subtypes

```
-- Ada derived types (TÜRETİLMİŞ TİP):  
type Celsius is new Float;  
type Fahrenheit is new Float;  
  
-- Celsius ve Fahrenheit birbirleriyle DEĞİL denkdir  
-- (aynı yapıya sahip olsalar bile)  
  
-- Ada subtype (ALT TİP):  
subtype Small_type is Integer range 0..99;  
  
-- Small_type, Integer ile DENKTİR  
  
-- NOT: Literaller universal_real/universal_integer tipindedir  
-- bu yüzden 3.0 her floating-point tiple uyumludur
```

i C
TYPE
EQ

C'nin Yaklaşımı: struct, enum ve union için **isim denkliđi** kullanır. Diđer nonscalar tipler için yapı denkliđi. typedef yeni tip oluşturmaz, sadece mevcut tipe takma isim verir.

Chapter 12: Review Questions - Tüm Sorular ve Çözümleri

S1. 1. Descriptor (tanımlayıcı) nedir?

✓ **ANS** Descriptor, bir değişkenin tüm özelliklerinin toplamıdır. Bellekte, değişkenin özelliklerini saklayan bir alandır. Statik özellikler için derleme zamanında (symbol table'ın parçası), dinamik özellikler için çalışma zamanında tutulur. Tip denetimi ve bellek tahsisi/serbest bırakma işlemleri için kullanılır.

S2. 2. Ondalık (decimal) veri tiplerinin avantaj ve dezavantajları nelerdir?

✓ **ANS** **WER** AVANTAJLAR: Ondalık değerleri tam olarak temsil edebilir (0.1 float'ta temsil edilemez ama decimal'de edilir); iş uygulamaları için idealdir. DEZAVANTAJLAR: Üs (exponent) kullanılamaz, değer aralığı kısıtlıdır; BCD formatı bellek israfeder (6 rakam: 24 bit decimal vs 20 bit binary); donanım desteği olmadığında yazılımda simüle edilmesi gerekir.

S3. 3. Karakter dizisi tipleri için tasarım soruları nelerdir?

✓ **ANS** **WER** 1) Stringler özel bir karakter dizisi mi yoksa ilkel tip mi olmalı? 2) Stringlerin uzunluğu statik mi yoksa dinamik mi olmalı?

S4. 4. Üç string uzunluğu seçeneğini açıklayın.

✓ **ANS** **WER** 1) STATIC LENGTH (Statik): Oluşturulduğunda uzunluk sabit. Python, Java String. 2) LIMITED DYNAMIC (Sınırlı Dinamik): Max uzunluk tanımda belirlenir. C, C-style C++. 3) DYNAMIC LENGTH (Dinamik): Maksimum yok, heap'te büyür/küçülür. JavaScript, Perl.

S5. 5. Ordinal, enumeration ve subrange tiplerini tanımlayın.

✓ **ANS** **WER** ORDINAL TİP: Değerler kümesi kolayca belirlenebilen ve sıralı olan tip. ENUMERATION TİP: Olası tüm değerlerin adlandırılmış sabitler olarak listelendiği tip. SUBRANGE TİP: Bir ordinal tipin sıralı alt kümesi.

S6. 6. Kullanıcı tanımlı numaralandırma tiplerinin avantajları nelerdir?

✓ **ANS** **WER** 1) OKUNABİLİRLİK: İsimlendirilmiş değerler kolayca anlaşılır. 2) GÜVENİLİRLİK: (a) Aritmetik işlemler yasak (günler toplanamaz). (b) Numaralandırma değişkenine tanımlı aralık dışında değer atanamaz.

S7. 7. C#'taki user-defined enumeration types, C++'takine göre nasıl daha güvenilirdir?

✓ **ANS** **WER** C#'ta numaralandırma değerleri tam sayıya coerce edilmez; aritmetik işlemler yasak. Değer aralığı tanımlı numaralandırma tipiyle kısıtlıdır. C++'ta ise numaralandırma değerleri tam sayıya otomatik dönüştürülebilir (myColor++) ve atama için cast gerekmez (belirli koşullarda).

S8. 8. Diziler için tasarım soruları nelerdir?

✓ **ANS
WER**

1) Alt simge (subscript) için hangi tipler geçerlidir? 2) Alt simge aralığı kontrol edilir mi? 3) Alt simge aralıkları ne zaman bağlanır? 4) Dizi tahsis ne zaman gerçekleşir? 5) Çentikli (jagged) mi yoksa dikdörtgen (rectangular) mi çok boyutlu diziler? 6) Depolama tahsisinde başlangıç değeri verilebilir mi? 7) Hangi dilimler (slices) desteklenir?

S9. 9. Static, fixed stack-dynamic, fixed heap-dynamic ve heap-dynamic dizileri tanımlayın. Her birinin avantajları nelerdir?

✓ **ANS
WER**

STATIC: Alt simge aralığı ve depolama statik. Avantaj: Yüksek verimlilik (dinamik tahsis yok). C/C++ static dizileri. FIXED STACK-DYNAMIC: Alt simge statik, tahsis çalışma zamanında stack'te. Avantaj: Alan verimliliği (farklı subprogramlar aynı alanı paylaşabilir). C fonksiyon dizileri. FIXED HEAP-DYNAMIC: Alt simge ve depolama kullanıcı isteğiyle heap'te tahsis. Avantaj: Esneklik (boyut probleme göre). Java, C# new[]. HEAP-DYNAMIC: Alt simge aralığı ve depolama tamamen değişebilir. Avantaj: Maksimum esneklik. Perl, Python, Ruby dizileri.

S10. 10. Perl'de var olmayan bir dizi elemanına erişildiğinde ne olur?

✓ **ANS
WER**

Perl'de var olmayan bir dizi elemanına erişildiğinde undef döner, ancak hata rapor edilmez.

S11. 11. JavaScript seyrek dizileri (sparse arrays) nasıl destekler?

✓ **ANS** JavaScript'te dizi elemanlarının alt simge değerlerinin sürekli (contiguous) olması gerekmez.
WER Örneğin 10 elemanlı `list[50]=42` ataması yapılırsa, list 11 eleman içerir ve uzunluk 51 olur. 11..49 arası elemanlar tanımlanmamış (undefined) sayılır ve bellekte yer kaplamaz.

S12. 12. Negatif alt simgeleri destekleyen diller hangileridir?

✓ **ANS** Ruby ve Perl negatif alt simgeleri destekler. Negatif indeks dizinin sonundan sayar. Python
WER negatif alt simgeleri desteklemez (Python bu konuda farklı bir mekanizma kullanır - slice notasyonu ile çalışır). JavaScript negatif alt simgeleri desteklemez.

S13. 13. Adım boyutu (stepsize) ile dizi dilimlerini destekleyen diller hangileridir?

✓ **ANS** Python: `vector[0:7:2]` → her ikinci elemanı alır. Perl de benzer dilim desteği sunar.
WER

S14. 14. Toplu sabit (aggregate constant) nedir?

✓ **ANS** Toplu sabit, depolama tahsisinde bir diziye başlatmak için kullanılan değerler listesidir.
WER Örnek: `int list[] = {4, 5, 7, 83};` C'de bu dizi 4 elemanla başlatılır.

S15. 15. Row major order ve column major order'ı tanımlayın.

✓ **ANS** ROW MAJOR ORDER: Dizinin ilk indeksinin alt sınır değerindeki elemanlar önce saklanır;
WER matris satır satır bellekte saklanır. C, C++, Java, Python kullanır. COLUMN MAJOR ORDER: Matris sütun sütun bellekte saklanır. Fortran ve MATLAB kullanır.

S16. 16. Bir dizi için erişim fonksiyonu (access function) nedir?

✓ **ANS** Dizi adı ve alt simge değerleri kümesinin, o elemanın bellek adresine eşlemesidir. Tek
WER boyut: `address(list[k]) = address(list[0]) + k * element_size`. Bu hesaplama çalışma zamanında yapılır; derleme zamanında önceden hesaplanamaz.

S17. 17. Java dizi descriptor'ında hangi girdiler gereklidir ve ne zaman saklanmalıdır?

✓ **ANS** Java dizileri fixed heap-dynamic olduğundan, descriptor çalışma zamanında saklanmalıdır.
WER İçerdiği bilgiler: eleman tipi, boyut sayısı, alt simge aralıkları ve dizinin başlangıç adresi.

S18. 18. İlişkilendirici dizinin (associative array) yapısı nedir?

✓ **ANS** İlişkilendirici dizi, anahtar-değer (key-value) çiftlerinden oluşan sırasız bir koleksiyondur.
WER Her eleman iki parçadan oluşur: anahtar (genellikle string) ve değer. Hash fonksiyonu kullanılarak elemanlara erişilir.

S19. 19. COBOL kayıtlarında düzey numaralarının (level numbers) amacı nedir?

✓ **ANS** Düzey numaraları, kayıt tanımının hiyerarşik yapısını gösterir. Göreceli değerlere göre:
WER daha yüksek düzey numarası = daha iç içe kayıt. 01: dış kayıt, 02-05+: iç alanlar veya alt kayıtlar.

S20. 20. Kayıt alanlarına tam nitelikli (fully qualified) ve eliptik (elliptical) referansları tanımlayın.

✓ **ANS
WER**

FULLY QUALIFIED REFERENCE: Alanın tüm kapsayan kayıt isimleri belirtilir. Örnek: FIRST OF EMPLOYEE-NAME OF EMPLOYEE-RECORD. ELLIPTICAL REFERENCE: Alan ismi verilir; bazı kapsayan kayıt isimleri atlanabilir. Sonuç referans benzersiz olduğu sürece geçerlidir. Örnek: FIRST (eğer belirsizlik yoksa).

S21. 21. Kayıt (record) ile tuple arasındaki temel fark nedir?

✓ **ANS** Tuple'da elemanların isimleri yoktur; elemanlara pozisyon (indeks) ile erişilir. Kayıtta her
WER alan adlandırılmıştır ve dot notation ile erişilir.

S22. 22. Python'un tuple'ları mutable midir?

✓ **ANS** Hayır. Python tuple'ları immutable'dır (değiştirilemez). Değiştirmek için list() ile diziye
WER çevrilmeli, değişiklik yapılmalı, ardından tuple() ile geri dönüştürülmelidir.

S23. 23. F#'taki tuple pattern'ın amacı nedir?

✓ **ANS** Tuple pattern, let ifadesinin sol tarafında yer alan bir isim dizisidir. Çoklu atama sağlar: let
WER a, b, c = tup → tup'ın elemanları sırasıyla a, b, c'ye atanır.

S24. 24. Esas olarak hangi imperatif dilde listeler dizi olarak da kullanılır?

✓ **ANS** Python'da listeler (list tipi) dizi işlevi de görür. Python'un dinamik dizileri aslında listelerdir.
WER

S25. 25. Scheme'deki CAR fonksiyonunun işlevi nedir?

✓ **ANS** CAR, liste parametresinin ilk elemanını döndürür. Örnek: (CAR '(A B C)) → A
WER

S26. 26. F#'daki tl fonksiyonunun işlevi nedir?

✓ **ANS** List.tl (tail), listenin ilk elemanı hariç geri kalanını döndürür. Örnek: List.tl [5; 7; 9] → [7; 9]
WER

S27. 27. Scheme'deki CDR fonksiyonu parametresini nasıl değiştirir?

✓ **ANS** CDR fonksiyonu parametresini değiştirmez; parametrenin ilk elemanı dışındaki geri kalan
WER listeyi döndürür. Örnek: (CDR '(A B C)) → (B C)

S28. 28. Python'un list comprehension'ları neye dayalıdır?

✓ **ANS** Python list comprehension'ları küme teorisi notasyonundan türetilmiştir. İlk olarak
WER fonksiyonel dil Haskell'de ortaya çıkmıştır.

S29. 29. Union, free union ve discriminated union'ı tanımlayın.

✓ **ANS** UNION: Değişkenin farklı zamanlarda farklı tiplerde değer tuttuğu bir tiptir. FREE UNION: Tip
WER denetimi yapılmayan union (C, C++). DISCRIMINATED UNION: Mevcut tipin hangi tip olduğunu belirten etiket (tag/discriminant) içeren union (ML, Haskell, F#, ALGOL 68).

S30. 30. F#'ın union'ları discriminated midir?

✓ **ANS** Evet. F#'ta union'lar discriminated union'dır. Her union değeri bir constructor ile oluşturulur
WER ve match ifadesi ile tip güvenli erişim sağlanır.

S31. 31. İşaretçi tipleri için tasarım soruları nelerdir?

✓ **ANS** 1) Pointer değişkeninin kapsamı ve yaşam süresi nedir? 2) Heap-dynamic değişkenin yaşam süresi ne kadardır? 3) Pointerlar belirli bir tipe kısıtlanmış mı? 4) Dinamik depolama, dolaylı adresleme veya her ikisi için mi? 5) Dil pointer tipini, referans tipini ya da her ikisini desteklemeli mi?

S32. 32. Pointer'larla ilgili iki yaygın problem nedir?

✓ **ANS** 1) DANGLING POINTER (Sarkan İşaretçi): Deallocate edilmiş bir heap değişkenine işaret eden pointer. 2) LOST HEAP-DYNAMIC VARIABLE (Bellek Sızıntısı): Tahsis edilmiş ama artık erişilemeyen heap değişkeni (garbage/çöp).

S33. 33. Neden çoğu dildeki pointerlar tek bir tipe işaret etmekle kısıtlanmıştır?

✓ **ANS** Tip denetimi (type checking) yapılabilmesi için. Farklı tiplere işaret edebilen pointer'ların dereference işleminde hangi tipin kullanılacağı bilinmez, bu da tip hatalarına yol açar.

S34. 34. C++ referans tipi nedir ve yaygın kullanımı nedir?

✓ **ANS** C++ referans tipi, daima örtük olarak dereference edilen sabit bir pointer'dır. Tanımlanırken bir değişkenin adresiyle başlatılmalıdır ve sonradan değiştirilemez. Yaygın kullanımı: fonksiyon tanımlarında formal parametre olarak (iki yönlü iletişim).

S35. 35. Formal parametre için C++ referans değişkenleri neden pointer'lardan daha iyidir?

✓ **ANS** Referans parametreleri, çağrılan fonksiyonda diğer parametreler gibi kullanılır; explicit dereference gerekmez. Bu kod okunabilirliğini artırır ve güvenliği iyileştirir. Derleyici adres iletimini otomatik yapar.

S36. 36. Java ve C# referans tipi değişkenleri, diğer dillerdeki pointer'lara göre hangi avantajlara sahiptir?

✓ **ANS** Java: Tüm sınıf örnekleri referansla erişilir; pointer aritmetiği yok; implicit deallocation (GC) sayesinde dangling reference oluşmaz. C#: Java referanslarına ek olarak pointer'lar da desteklenir ancak unsafe olarak işaretlenmeli. Her iki dilde de referansla erişilen nesneler implicit olarak deallocate edilir.

S37. 37. Garbage toplama için lazy ve eager yaklaşımları açıklayın.

✓ **ANS** EAGER (Reference Counter): Çöp oluştuğu artımlı (incremental) olarak toplanır. Her pointer değişiminde sayaç güncellenir. Avantaj: Büyük duraklamalar yok. LAZY (Mark-Sweep): Heap dolduğunda tetiklenir. Tüm heap taranır. Avantaj: Basit uygulama. Dezavantaj: Büyük duraklamalar.

S38. 38. Java ve C# referanslarında aritmetik neden anlamsızdır?

✓ **ANS** Referans değişkeni bellek adresini değil, bir nesneyi/değeri referans eder. Nesne üzerinde aritmetik işlem yapmanın anlamı yoktur. Pointer'lar adres tutar, bu yüzden pointer aritmetiği mantıklıdır; referanslar nesne tutar, aritmetik anlamsız.

S39. 39. Uyumlu tip (compatible type) nedir?

✓ **ANS** Uyumlu tip; bir operatör için ya yasal olan ya da örtük dönüşüm (coercion) ile yasal hale getirilebilen tiptir.

S40. 40. Tip hatası (type error) nedir?

✓ **ANS**
WER

Bir operatörün uygunsuz tip operanda uygulanmasıdır. Örneğin: C'nin eski sürümünde int değer float beklenen fonksiyona geçirilmesi.

S41. 41. Güçlü tipli (strongly typed) nedir?

✓ **ANS** Tip hatalarının her zaman tespit edildiği bir programlama dilidir. Tüm operandların tipleri
WER derleme ya da çalışma zamanında belirlenebilmelidir.

S42. 42. Java neden tam anlamıyla strongly typed değildir?

✓ **ANS** Java'da explicit cast (dönüşüm) yapılabilir ve bu tip hatalarına yol açabilir. Ayrıca coercion
WER mevcuttur (int + float = float). Ancak C/C++'a göre çok daha güvenlidir.

S43. 43. Dönüştürmeyen cast (non-converting cast) nedir?

✓ **ANS** Bir değeri farklı bir tip olarak yorumlayan cast işlemidir; değerin kendisi değişmez. Örneğin
WER C'de int değeri float pointer'ıyla dereference etmek.

S44. 44. Tip coercion olmayan diller hangileridir?

✓ **ANS** ML ve F# coercion içermez; en güvenilir tip sistemleri bunlardır. Ada da çok az coercion
WER içerir (universal tipler hariç).

S45. 45. C ve C++ neden strongly typed değildir?

✓ **ANS** Her iki dil de union tiplerini içerir ve bu union'lar tip denetimi yapılmaz (free union). Bunun
WER yanı sıra C ve C++ void* pointer ve C-style cast ile tip sistemini atlayabilir.

S46. 46. İsim denkliği (name type equivalence) nedir?

✓ **ANS** Aynı bildirimde ya da aynı tip ismini kullanan bildirimlerde tanımlanan iki değişkenin
WER eşdeğer tipte sayıldığı denklik kuralıdır.

S47. 47. Yapı denkliği (structure type equivalence) nedir?

✓ **ANS** İki değişkenin tiplerinin özdeş yapıya sahipse eşdeğer kabul edildiği denklik kuralıdır.
WER

S48. 48. İsim denkliğinin temel avantajı nedir?

✓ **ANS** Uygulaması kolaydır: yalnızca iki tip ismi karşılaştırılır. Aynı yapıdaki farklı semantik tiplerin
WER (Celsius vs Fahrenheit) karıştırılmasını önler.

S49. 49. Yapı denkliğinin temel dezavantajı nedir?

✓ **ANS** Aynı yapıya sahip ama farklı anlama gelen tiplerin ayırt edilmesine izin vermez. Örnek: type
WER Celsius = Float; type Fahrenheit = Float; — yapı denkliğinde bunlar eşdeğer!

S50. 50. C hangi tipler için yapı denkliği kullanır?

✓ **ANS** C, array tipleri için yapı denkliği kullanır. struct, enum ve union için ise isim denkliği kullanır
WER (farklı dosyalardaki aynı yapılar hariç — bu durumda yapı denkliği uygulanır).

S51. 51. C'nin struct veri tipini hangi küme işlemi modeller?

✓ **ANS**
WER

Kartezyen çarpım (Cartesian product). struct intFloat { int myInt; float myFloat; }; → int * float kartezyen çarpımını tanımlar. Eleman isimleri (myInt, myFloat) saf matematiksel modelde yoktur.

Chapter 13: Problem Set - Tüm Sorular ve Çözümleri

P1. Boolean değerlerin bellekte tek bit olarak tutulması için argümanlar nelerdir?

✓ **ANS**
WER

LEHTE: Teorik olarak minimum alan kullanımı (1 bit). ALEYHİNE: Birçok makinede tek bir bit verimli şekilde adreslenemez; bu nedenle pratikte 1 byte kullanılır. Bit düzeyinde erişim için maskeleme gerekir, bu da yavaş ve karmaşıktır. Sonuç: Modern sistemlerde Boolean için 1 byte kullanmak hem pratik hem de performanslıdır.

P2. Decimal (ondalık) değer belleği nasıl israf eder?

✓ **ANS**
WER

BCD formatında her ondalık rakam için en az 4 bit gereklidir. 6 basamaklı ondalık sayı: 24 bit (packed BCD) veya 48 bit (unpacked). Aynı sayı binary'de: 20 bit yeterlidir. Bunun yanı sıra ondalık işlemler için özel donanım gerekir; bu da hem alan hem de zaman açısından maliyet getirir.

P4. Tombstone ve locks-and-keys yöntemlerini güvenlik ve uygulama maliyeti açısından karşılaştırın.

✓ **ANS**
WER

GÜVENLİK: Her ikisi de dangling pointer'ları tespit eder. Tombstone: pointer'ın deallocate edilen değişkene erişimini engeller. Locks-and-keys: her erişimde key==lock kontrolü yapar, alias'lar da korunur. UYGULAMA MALİYETİ: Tombstone: tombstone'lar hiç deallocate edilmez (bellek birikir) + ek indirection. Locks-and-keys: her pointer değeri (key,address) çifti; her erişimde karşılaştırma. SONUÇ: Her ikisi de pahalı; bu yüzden hiçbir yaygın dil bunları kullanmaz. En iyi çözüm implicit deallocation (garbage collection)'dır.

P5. Pointer'ların yalnızca belirli bağlamlarda örtük (implicit) dereference edilmesinin dezavantajları nelerdir?

✓ **ANS**
WER

1) Tutarsız sözdizimi: Bazen explicit *, bazen otomatik dereferencing. Programcı hangi bağlamda ne olduğunu takip etmek zorundadır. 2) Okunabilirlik azalır: Kodu okuyan kişi dereferencing'in gerçekleşip gerçekleşmediğini anlamak için bağlamı bilmek zorundadır. 3) Hata ayıklama güçlüğü: Örtük dereferencing beklenmedik sonuçlar doğurabilir.

P6. Ada'nın subtype'ları ve derived type'ları arasındaki tüm farklılıkları açıklayın.

✓ **ANS**
WER

DERIVED TYPE: Var olan tipten bağımsız yeni bir tip oluşturur. Ana tipin özellikleri (operasyonlar) miras alınır ama tip uyumluluğu yoktur. Range kısıtlaması eklenebilir. Literaller (3.0 gibi) evrensel tiptedir, dönüşüm sorunsuz. SUBTYPE: Mevcut tipin muhtemelen aralık kısıtlı versiyonudur. Üst tipiyle TÜM AÇILARDAN TIP UYUMLU. Aynı değişkenler karışık kullanılabilir. ÖZET: Derived type → farklı tip; Subtype → aynı tip, kısıtlı aralık.

P7. C ve C++'daki -> operatörü için önemli gerekçe nedir?

✓ **ANS**
WER

-> operatörü, bir struct pointer'ının alanına erişimi kolaylaştırır. (*p).age ifadesini p->age şeklinde kısaltır. Önce explicit dereference (*p) sonra alan erişimi (.) yazmak hata yapma ihtimalini artırır ve okunabilirliği azaltır. -> bu iki işlemi tek, anlaşılır bir operatörde birleştirir.

P8. C++ ve Java'nın numaralandırma tiplerindeki tüm farklılıklar nelerdir?

✓ **ANS
WER**

C++: (a) Değerler tam sayıya otomatik dönüştürülür; (b) myColor++ gibi işlemler geçerli; (c) cast ile sayısal değer atanabilir; (d) Sadece tam sayı değeri olabilir. JAVA 5+: (a) Tüm enum tipleri Enum sınıfının alt sınıfı; (b) Alanlar, constructor ve metodlar içerebilir; (c) Hiçbir zaman tam sayıya coerce edilmez; (d) toString, values(), ordinal() metodları miras alınır; (e) Başka tipten atama yapılamaz.

P9. 3 boyutlu diziler için row major ve column major erişim fonksiyonlarını geliştirin.

✓ **ANS
WER**

3 boyutlu dizi $a[i][j][k]$, boyutlar $n_1 \times n_2 \times n_3$ olsun (lower bounds = 0): ROW MAJOR (C tarzı): $\text{location}(a[i,j,k]) = \text{base} + (i \cdot n_2 \cdot n_3 + j \cdot n_3 + k) \cdot \text{element_size}$ COLUMN MAJOR (Fortran tarzı): $\text{location}(a[i,j,k]) = \text{base} + (k \cdot n_1 \cdot n_2 + j \cdot n_1 + i) \cdot \text{element_size}$ Generik (lower bounds lb_1, lb_2, lb_3 ile): Row Major: $\text{base} + ((i-lb_1) \cdot n_2 \cdot n_3 + (j-lb_2) \cdot n_3 + (k-lb_3)) \cdot \text{element_size}$

Chapter 14: Pratik Quiz Soruları

Practice Quiz Questions – Quiz Tarzında Hazırlanmış

BİLGİ

Bu bölümdeki sorular, dersteki Quiz 1-7'ye benzer formatta hazırlanmıştır. Her soru Part A (teorik) + Part B (uygulama) şeklindedir. Cevaplar hemen altında verilmiştir.

PRACTICE QUIZ A

PAR T A

PART A: Compile-time'da hata tespiti mi yoksa run-time'da mı daha iyidir? Avantaj ve dezavantajlarını tartışın.

✓ ANS WER

COMPILE-TIME (Derleme Zamanı) Hata Tespiti: AVANTAJLAR: (1) Hataları erken yakalamak daha az maliyetlidir. (2) Çalışan programa zarar vermez. (3) Tip güvenliği sağlanır (strongly typed). (4) Programcıya daha iyi geliştirme deneyimi sunar. DEZAVANTAJLAR: (1) Programcı esnekliği azalır. (2) Bazı hatalar yalnızca çalışma zamanında belli olur (kullanıcı girdisine bağlı). (3) Derleme süresi uzayabilir. RUN-TIME (Çalışma Zamanı) Hata Tespiti: AVANTAJLAR: (1) Daha fazla esneklik. (2) Bazı dinamik durumlar yakalanabilir. (3) Yorumlanan diller için kaçınılmazdır. DEZAVANTAJLAR: (1) Hata tespit edildiğinde program çökmüş ya da zarar vermiş olabilir. (2) Performans düşer (sürekli tip denetimi). (3) Her senaryoyu test etmek zor.

PAR T B

PART B: C'de orthogonality (ortogonallik) özelliğine ve ortogonal olmayan özelliğe birer örnek verin.

✓ ANS WER

ORTHOGONAL ÖZELLİK (C): Pointer tipleri herhangi bir temel tipiyle birleştirilebilir (int*, float*, char* vb.). Bu birleşim tutarlı ve bağımsız davranır → orthogonality. ORTHOGONAL OLMAYAN ÖZELLİK (C): void tipindeki bir değişken tanımlanamaz (void x; geçersiz), ancak void* pointer tanımlanabilir. Ayrıca: struct içinde fonksiyon tanımlanamaz (C'de). Bir diğer örnek: diziler (arrays) fonksiyonun dönüş değeri olamaz, ama struct olabilir. Bunlar ortogonallik ihlalleridir.

PRACTICE QUIZ B

PAR T A

PART A: Java hem derleme hem de JIT (Just-In-Time) derleme kullanır. Bu ikisi arasındaki farkı açıklayın.

✓ ANS WER

GELENEKSEL DERLEME (Compilation): Java kaynak kodu (.java) → Bytecode (.class) dönüştürülür. Bu bytecode platform bağımsızdır. Java Virtual Machine (JVM) bytecode'u çalıştırır. Bytecode, JVM'nin yorumlayıcısı tarafından satır satır yürütülür. JIT (Just-In-Time) DERLEME: JVM, sık çalıştırılan bytecode bloklarını çalışma zamanında yerel makine koduna derler. Bu, yorumlama overhead'ini ortadan kaldırır. Sıcak noktalar (hot spots) optimize edilir. İlk çalıştırma daha yavaş olabilir ama sonraki çalıştırmalar hızlıdır. SONUÇ: Java, taşınabilirlik (bytecode) ve performans (JIT) dengesini bu şekilde kurar.

**PAR
T B**

PART B: Aşağıdaki C ifadesi için lexical analiz ve syntax analiz adımlarını gösterin: $Z = A + B * 2;$

**✓ ANS
WER**

LEXICAL ANALİZ (Token'lara Ayırma): $Z \rightarrow \text{ID (identifier)} = \rightarrow \text{ASSIGN_OP}$ $A \rightarrow \text{ID}$ $+ \rightarrow \text{ADD_OP}$ $B \rightarrow \text{ID}$ $* \rightarrow \text{MULT_OP}$ $2 \rightarrow \text{INT_LITERAL}$; $\rightarrow \text{SEMICOLON}$ SYNTAX ANALİZ (Parse Tree): $\rightarrow = \rightarrow Z = \rightarrow Z = + \rightarrow Z = + \rightarrow Z = A + * \rightarrow Z = A + B * 2$

PRACTICE QUIZ C

**PAR
T A**

PART A: 1950'lerin başında yorumlama (interpretation) yavaşlığı neden kabul edilebilirdi? 1950'lerin ortasında neden kabul edilemez hale geldi?

**✓ ANS
WER**

1950'LERİN BAŞI - KABUL EDİLEBİLİR: İlk bilgisayarlar (1950'ler başı) makine diliyle programlanıyordu. Assembler bile büyük kolaylıktı. Herhangi bir üst düzey dil devrimciydi. Bilgisayarlar çok pahalıydı ve az kişi kullanıyordu; yorumlama yavaşlığı, programlama kolaylığının yanında önemsiz görünüyordu. Donanım zaten yavaştı, ek overhead fark yaratmıyordu. 1950'LERİN ORTASI - KABUL EDİLEMEZ: Fortran'ın tanıtılmasıyla (1957) derleme ile yorumlama arasındaki hız farkı açıkça hissedilmeye başlandı. Bilimsel hesaplamalar için verimlilik kritikti. IBM 704 gibi makinelerde derlenmiş kod 10-100 kat daha hızlıydı. Programcılar ve kurumlar bu farkı artık göz ardı edemezdi.

**PAR
T B**

PART B: Typeless (tipsiz) programlama diline bir örnek verin. Tipsiz dilin lehinde ve aleyhinde argümanlar nelerdir?

**✓ ANS
WER**

TİPSİZ DİL ÖRNEĞİ: BCPL, Bliss ya da erken sürüm C (tipsiz kabul edilebilir). Günümüzden örnek: Assembly dili (tipler programcı sorumluluğundadır). LEHİNDE ARGÜMANLAR: (1) Maksimum esneklik: herhangi bir değer herhangi bir değişkende tutulabilir. (2) Küçük ve basit dil implementasyonu. (3) Düşük seviyeli programlama için uygun. (4) Tip dönüşüm overhead'i yok. ALEYHİNDE ARGÜMANLAR: (1) Tip hataları yakalanmaz; güvenilirlik çok düşük. (2) Okunabilirlik azalır; kodun ne yaptığı belirsizleşir. (3) Derleyici optimizasyonu yapamaz. (4) Büyük projelerde bakım zorlaşır.

PRACTICE QUIZ D

**PAR
T A**

PART A: $n > 0$ olmak üzere n adet 'a' harfinden sonra n adet 'b' harfi gelen stringleri üreten bir gramer yazın. (ab, aabb, aaabbb geçerli; a, abb, ba geçersiz)

**✓ ANS
WER**

GRAMER: $S \rightarrow a S b \mid a b$ ya da eşdeğer olarak: $S \rightarrow a b \mid a S b$ AÇIKLAMA: Bu gramer, Context-Free Grammar (CFG). Temel durum: $S \rightarrow ab$ (1 tane a, 1 tane b). Özyinelemeli durum: $S \rightarrow aSb$ (her özyinelemede 1 a ve 1 b eklenir). Böylece n adet a, n adet b garanti edilir ve $n > 0$ koşulu sağlanır. DOĞRULAMA: ab: $S \rightarrow ab$ ✓ aabb: $S \rightarrow aSb \rightarrow a(ab)b = aabb$ ✓ aaabbb: $S \rightarrow aSb \rightarrow a(aSb)b \rightarrow aa(ab)bb = aaabbb$ ✓

**PAR
T B**

PART B: Aşağıdaki BNF'i (1) prefix ++ ve -- unary operatörlerini ekleyecek şekilde ve (2) + operatörünü sağdan ilişkilendirici (right associative) yapacak şekilde yeniden yazın. ++ ve -- en yüksek önceliğe sahip olmalıdır. BNF: $\rightarrow + \mid - \mid \rightarrow * \mid / \mid \rightarrow ** \mid \rightarrow () \mid id$

**✓ ANS
WER**

YENİDEN YAZILMIŞ BNF (++ ve -- en yüksek öncelik, + sağdan ilişkilendirici): $\rightarrow + \mid - \mid [NOT: + \text{ için sağ tarafta kullanarak sağdan ilişkilendirme sağlandı}] \rightarrow * \mid / \mid \rightarrow ** \mid \rightarrow ++ \mid -- \mid [NOT: ++ \text{ ve } -- \text{ en yüksek öncelik için 'den hemen önce eklendi}] \rightarrow () \mid id$ NEDEN SAĞDAN İLİŞKİLENDİRME? Sol ilişkilendirmede: $\rightarrow +$ Sağ ilişkilendirmede: $\rightarrow +$

PRACTICE QUIZ E**PAR
T A**

PART A: Aşağıdaki gramerin dolaylı sol-özyinelemeli (indirect left-recursive) olduğunu gösterin ve sol özyinelemeyi ortadan kaldırın: $A \rightarrow B a \mid c B \rightarrow A b \mid d$

**✓ ANS
WER**

DOLAYLI SOL ÖZYİNELEME KANITLANMASI: $A \rightarrow B a \mid B \rightarrow A b \rightarrow (A \rightarrow B a) b \rightarrow B a b$
Dolayısıyla: $A \rightarrow B a \rightarrow (A b) a \rightarrow A b a$, dolaylı olarak kendisiyle başlamaktadır $\rightarrow$ Indirect left recursion ✓ SOL ÖZYİNELEME GİDERME: Adım 1: A'yı B kuralına koy: $B \rightarrow A b \mid d \rightarrow B a b \mid c b \mid d$ Adım 2: B'deki doğrudan sol özyineleme ($B \rightarrow B a b$): $B \rightarrow c b B' \mid d B' B' \rightarrow a b B' \mid \epsilon$ Adım 3: Yeni A kuralları: $A \rightarrow B a \mid c A \rightarrow (c b B' \mid d B') a \mid c A \rightarrow c b B' a \mid d B' a \mid c$ SONUÇTAKİ GRAMER: $A \rightarrow c b B' a \mid d B' a \mid c B \rightarrow c b B' \mid d B' B' \rightarrow a b B' \mid \epsilon$

**PAR
T B**

PART B: Tip denetimi (type checking) neden önemlidir? Statik ve dinamik tip denetimini karşılaştırın.

**✓ ANS
WER**

TİP DENETİMİNİN ÖNEMİ: Hata tespiti: tip uyumsuzluklarını erken yakalamak programların güvenilirliğini artırır. Belgeleme: tip bildirimleri programın veri yapısı hakkında bilgi verir. Optimizasyon: derleyici tip bilgisini kullanarak daha verimli kod üretebilir. STATİK TİP DENETİMİ: Derleme zamanında yapılır. Avantajlar: (1) Hatalar erken tespit edilir ve düzeltilmesi ucuzdur. (2) Çalışma zamanı overhead yok. (3) Daha iyi derleyici optimizasyonu. Dezavantajlar: (1) Esneklik kısıtlıdır. (2) Bazı hatalar yakalanmaz (runtime-only). DİNAMİK TİP DENETİMİ: Çalışma zamanında yapılır. Avantajlar: (1) Daha esnek. (2) Dinamik tiplendirme mümkün. Dezavantajlar: (1) Hata tespit geç olur. (2) Yavaş (sürekli tip kontrolü). (3) Test zorlaşır (her durum çalıştırılmalı).

PRACTICE QUIZ F**PAR
T A**

PART A: Aşağıdaki BNF için bir attribute grammar (öznitelik grameri) yazın. Dil kuralı: ifadelerde (expressions) veri tipleri karıştırılamaz, ancak atama deyimlerinde (assignment statements) her iki tarafın tipi aynı olmak zorunda değildir. $\rightarrow = \rightarrow [2] + [3] \mid \rightarrow A \mid B \mid C$

✓ ANSWER

ATTRIBUTE GRAMMAR: Kural 1: $\rightarrow =$ Semantic rule: $\text{.typeError} \leftarrow \text{.typeError}$ [NOT: Atamada tipler eşleşmek zorunda değil; sadece expr kendi içinde tutarlı olmalı] Kural 2: $\rightarrow [2] + [3]$ Semantic rule: $\text{.actualType} \leftarrow \text{if } [2].\text{actualType} == [3].\text{actualType} \text{ then } [2].\text{actualType} \text{ else type_error}$ Predicate: $[2].\text{actualType} == [3].\text{actualType}$ [NOT: İfade içinde tipler karıştırılmaz!] Kural 3: $\rightarrow$ Semantic rule: $\text{.actualType} \leftarrow \text{.actualType}$ Kural 4: $\rightarrow A$ Semantic rule: $\text{.actualType} \leftarrow \text{lookup}(A)$ Kural 5: $\rightarrow B$ Semantic rule: $\text{.actualType} \leftarrow \text{lookup}(B)$ Kural 6: $\rightarrow C$ Semantic rule: $\text{.actualType} \leftarrow \text{lookup}(C)$

PART B: Aşağıdaki atama deyimleri için en zayıf ön koşulu (weakest precondition) hesaplayın: (1) $a = 2 * (b - 1) - 1 \{a > 0\}$ (2) $b = (c + 10) / 3 \{b > 6\}$ (3) $a = a + 2*b - 1 \{a > 1\}$ (4) $x = 2*y + x - 1 \{x > 11\}$

EN ZAYIF ÖN KOŞUL HESAPLAMA: {Q} a = expr {P} → wp: P[a/expr] (1) a = 2*(b-1)-1 {a > 0} a yerine 2*(b-1)-1 koy: 2*(b-1)-1 > 0 2b-2-1 > 0 2b > 3 b > 3/2 → WP: b > 1.5 (ya da b >= 2 tam sayı için) (2) b = (c+10)/3 {b > 6} (c+10)/3 > 6 c+10 > 18 c > 8 → WP: c > 8 (3) a = a + 2*b - 1 {a > 1} a + 2*b - 1 > 1 a + 2b > 2 → WP: a + 2*b > 2 (4) x = 2*y + x - 1 {x > 11} 2*y + x - 1 > 11 2y + x > 12 → WP: 2*y + x > 12

PRACTICE QUIZ G

PART A: Aşağıdaki EBNF ile $a * (b + c)$ stringi için recursive descent parser izini (trace) gösterin: $\rightarrow \{ (+ | -) \} \rightarrow \{ (* | /) \} \rightarrow \text{id} \mid \text{int} \mid \text{constant} \mid ()$

RECURSIVE DESCENT PARSER İZİ: a * (b + c) expr() çağırısı: term() çağırısı: factor() çağırısı: Token: a → id olduğu için a eşleştirilir, token ilerletilir (*) return {(*/) factor} döngüsü: token = * → eşleşti, ilerlet factor() çağırısı: Token: (→ (eşleştirilir, ilerletilir (b) expr() çağırısı (iç): term() çağırısı: factor() çağırısı: Token: b → id, eşleştirilir (+) {(*/) factor} döngüsü: token = + → *, / değil, döngü biter return {(+|-) term} döngüsü: token = + → eşleşti, ilerlet term() çağırısı: factor() çağırısı: Token: c → id, eşleştirilir ()) {(*/) factor} döngüsü: token =) → döngü biter return {(+|-) term} döngüsü: token =) → döngü biter return) eşleştirilir → token biter (EOF) return {(*/) factor} döngüsü: EOF → döngü biter return {(+|-) term} döngüsü: EOF → döngü biter BAŞARI: Tüm input tüketildi ✓

PART B: Güçlü tiplendirme (strong typing) kavramını tanımlayın ve C, C++, Java, ML dillerinin güçlü tipli olup olmadığını değerlendirin. Coercion'ın güçlü tiplendirme üzerindeki etkisini açıklayın.

GÜÇLÜ TİPLEME (STRONG TYPING): Tüm tip hatalarının derleme ya da çalışma zamanında MUTLAKA tespit edildiği sistemdir. Hiçbir tip hatası sessizce geçemez. C: GÜÇLÜ TİPLİ DEĞİL. Free union (tip denetimi yok), void* pointer, implicit coercion, eski prototypesiz fonksiyonlar. C++: GÜÇLÜ TİPLİ DEĞİL. C'nin union'larını miras alır. Reinterpret_cast ile tip sistemi tamamen atlanabilir. Java: NEREDEYSE GÜÇLÜ TİPLİ. Explicit cast hata üretebilir (ClassCastException). int+float = float (coercion var). Ama C/C++'a göre çok daha güvenli. ML: TAM GÜÇLÜ TİPLİ. Coercion yok. Tüm tip hataları derleme zamanında yakalanır. En güvenilir tip sistemi. COERCION'IN ETKİSİ: Coercion, tip sisteminin bir güvenlik faydasını ortadan kaldırır. Java'da 'a + d' yazılması gereken yerde 'a + b' yazılırsa (d float, b int), int otomatik float'a dönüşür ve hata yakalanmaz. ML ve F# hiç coercion yapmaz → en güvenilir.

CSE 341 - Data Types

Kapsamlı Çalışma Rehberi

Bu PDF aşağıdakileri kapsamaktadır:

- ✓ Bölüm 6 Konu Anlatımı (Türkçe açıklamalar + İngilizce terimler)
 - ✓ Tüm Review Questions (51 soru) - Detaylı Cevaplar
 - ✓ Problem Set Çözümleri
- ✓ 7 Adet Pratik Quiz (Quiz A-G) - Quiz tarzında sorular
 - ✓ Tablolar, Kod Örnekleri, Karşılaştırmalar